

School LLM — Deployment & AI Infrastructure Plan

Document type: Internal planning document for leadership review

Reference scenario: One school, 100 teachers + 500 students

Cost basis: Public list pricing as of May 2026

Currency note: Costs shown in USD with INR equivalent at 1 USD ≈ ₹84

1. Executive Summary

The School LLM application needs an AI engine to power its features (PDF Q&A, summaries, quizzes, assignment grading). The AI engine can be sourced in several different ways — each with very different costs, quality levels, operational effort, and scaling behavior. This document surveys all the realistic approaches, explains how each one works in plain language, lays out the cost of every option, and concludes with a phased recommendation.

Recommendation in one sentence: start with a managed AI API for the first 1–2 schools, then transition to a hybrid setup — a rented GPU server (running an open-source AI model) handles the bulk of queries, with a managed API kept as a fallback for the most complex queries.

For the reference school (100 teachers + 500 students, ~100,000 AI calls per month), the five approaches compare as follows:

Approach	Cost per school per month	Quality	Data control	Operational effort
1. Managed AI API (Claude)	₹60,000	Highest	Vendor cloud	Lowest
2. Self-hosted on cloud GPU VM	₹17,000–61,000	Mid	Self-controlled	High
3. Self-hosted on bare metal	₹4,000–8,000 (after one-time hardware)	Mid	Self-controlled	Highest
4. Serverless GPU inference	₹15,000–80,000 (usage-driven)	Mid	Limited control	Medium
5. Hybrid (Recommended)	₹27,000 (1 school) → ₹16,000 (3 schools)	Near-best	Mostly self-controlled	Medium

2. Reference Workload

The cost numbers in this document assume one school of **100 teachers and 500 students** operating ~24 active days per month.

Activity assumptions

Population	Daily active %	LLM calls per active user per day
Teachers	60% (≈60 active)	8 (PDF upload, assignment grading, quiz generation, summary)
Students	50% (≈250 active)	15 (Q&A on materials, explanations, homework help)

Daily calls: $60 \times 8 + 250 \times 15 = 4,230$ calls

Monthly calls (24 active days): ≈100,000 LLM calls

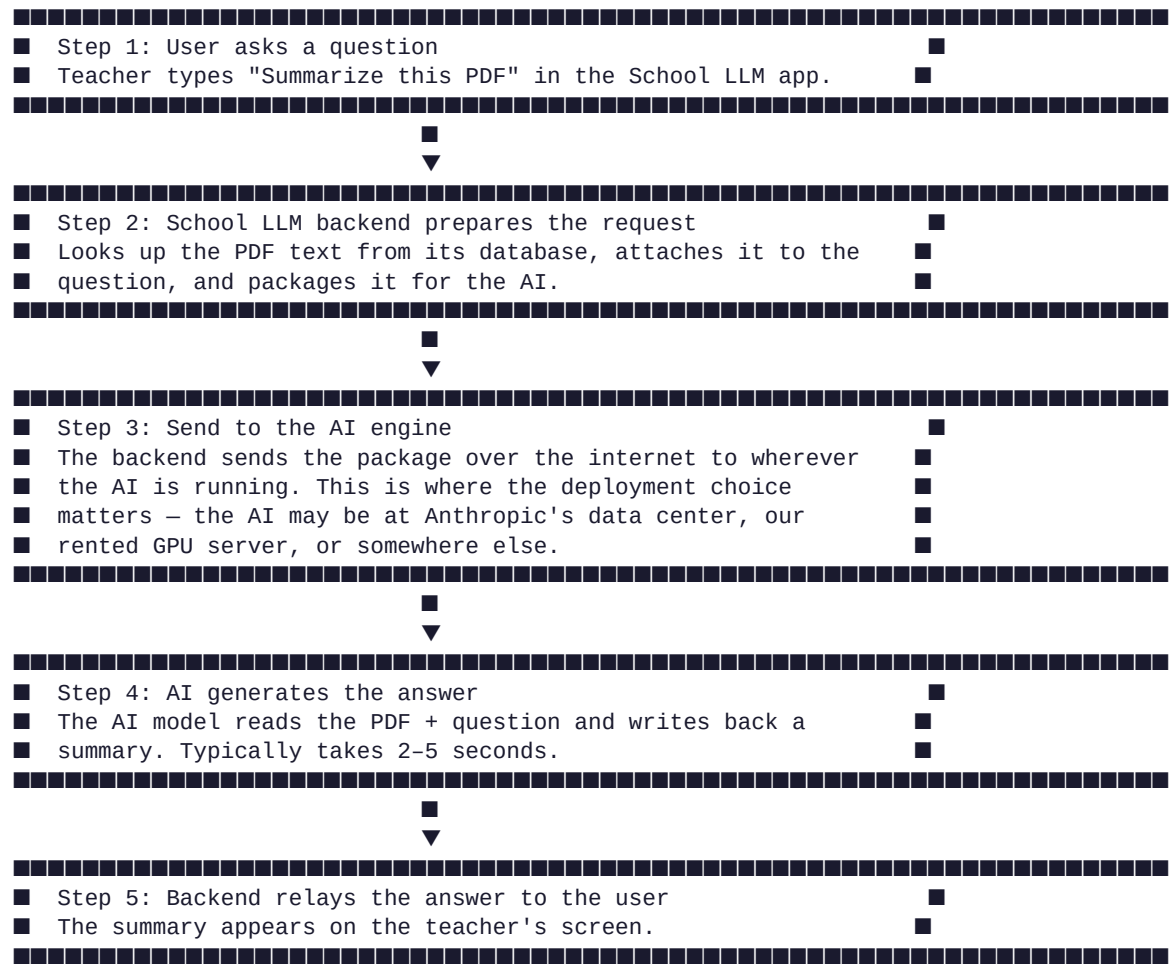
Token volume per call: 2,500 input + 600 output tokens

Monthly volume: 250 million input tokens + 60 million output tokens

Model mix: 70% lightweight tasks (Q&A, evaluation), 30% complex tasks (summaries, essay grading)

3. How AI Actually Powers the Application — Plain English Workflow

Before comparing deployment approaches, it helps to understand what actually happens when a teacher or student asks the AI something. This workflow is the same regardless of which approach is chosen — only the **box that runs the AI** changes.



Two important consequences:

- **The AI engine must be available 24/7.** Whether the AI runs on someone else's server (API) or our own server, it has to be ready to answer instantly. Cold starts (waiting for the AI to load) ruin the user experience.
- **One AI engine can serve many users simultaneously.** Modern AI software groups parallel questions together for efficiency. A single decent GPU can serve 30–50 concurrent questions before noticeably slowing down — which is enough for one school of this size.

4. Deployment Approaches — All Five Ways

There are five realistic ways to run the AI engine for the School LLM. The big choice is **who owns the hardware**: us, a cloud provider, an AI vendor, or a serverless platform. Below is each approach explained in plain language, with cost.

Approach 1 — Managed AI API (use someone else's AI as a service)

How it works in plain language:

We don't run any AI hardware. Every time a user asks something, our application makes an internet call to a third party (Anthropic, Google, OpenAI) who runs the AI on their own servers. We send the question, they send back the answer. We get billed per million tokens of input and output.

What we manage: The School LLM application code, the customer database, the user accounts. Nothing AI-related at the infrastructure level.

Pricing (May 2026):

Provider / Model	Input cost (per million tokens)	Output cost (per million tokens)	Notes
Anthropic Claude Haiku 4.5	\$1.00	\$5.00	Fast, good quality
Anthropic Claude Sonnet 4.6	\$3.00	\$15.00	Best quality
Anthropic Claude Opus 4.7	\$5.00	\$25.00	Highest tier (rarely needed)
Google Gemini 2.5 Flash	\$0.30	\$2.50	Cheaper, good quality
OpenRouter Llama 3.3 70B	\$0.10	\$0.32	Cheapest paid option

Per-school monthly cost (100K calls/month workload, Claude with 70/30 Haiku/Sonnet mix + caching enabled):

Component	Calculation	Cost
Claude Haiku input (cached)	175M tokens × \$0.575/M	\$101
Claude Haiku output	42M tokens × \$5/M	\$210
Claude Sonnet input (cached)	75M tokens × \$1.725/M	\$129
Claude Sonnet output	18M tokens × \$15/M	\$270
Total per school per month		\$710 ≈ █60,000

If we used Gemini Flash instead of Claude: ~█16,000/school/month

If we used the cheapest OpenRouter (Llama 3.3 70B): ~█4,000/school/month — but lower quality

Pros:

- Zero capital expenditure, no hardware to procure
- Highest-quality models available
- New schools onboarded in minutes — just give them an API key
- Auto-scales with usage; no capacity planning
- Vendor handles all infrastructure: uptime, security patches, model upgrades
- Best for fast experimentation

Cons:

- Cost grows linearly with users — every new school adds its full monthly cost
- Student data leaves our infrastructure (regulatory consideration for sensitive data)
- Vendor controls pricing and policies; potential lock-in
- Network dependency — if Anthropic has an outage, all our users are affected
- Hard to negotiate volume discounts at small scale

Approach 2 — Self-hosted Ollama on a Cloud GPU Virtual Machine

How it works in plain language:

We rent a virtual machine that has a GPU attached to it from a cloud provider (like AWS or E2E Networks). On that machine, we install Ollama (or vLLM) and download an open-source AI model (like Llama 3.1 8B). The model runs entirely on our rented GPU. Our School LLM application sends questions to this server over the internal network.

What we manage: The GPU virtual machine (provisioning, security, updates, monitoring), the AI model file, the inference software, plus everything the application needs.

Pricing — GPU provider comparison (always-on, 24/7):

Provider	Hardware	India region?	Monthly cost	Notes
E2E Networks (Indian)	L4 GPU (24 GB)	■ Mumbai / Delhi	■30,000 reserved / ■43,000 on-demand	Indian DC, INR billing, Indian support
AWS EC2 g6.xlarge	L4 (24 GB)	■ Mumbai	■49,000 on-demand	Hyperscaler reliability
AWS EC2 g5.xlarge	A10G (24 GB)	■ Mumbai	■61,000 on-demand / ■39,000 1-yr reserved / ■26,000 3-yr reserved	Most reliable
Google Cloud g2-standard-4	L4 (24 GB)	■ Mumbai	~■50,000 on-demand	Similar to AWS
Microsoft Azure NC4as_T4_v3	T4 (16 GB)	■ Pune	~■35,000 on-demand	Slightly less GPU memory
Hetzner GEX44	RTX 4000 Ada (20 GB)	■ Germany / Finland	■17,000/month + ■5,000 one-time setup	Cheapest. ~150 ms higher latency from India.
RunPod Secure Cloud	RTX 4090 (24 GB)	■ Multiple US/EU	■20,000/month	Cheap, on-demand
Vast.ai (marketplace)	RTX 4090 (24 GB)	■ Various	■16,000/month	Cheapest, but unreliable — not for production
Lambda Labs	A10G	■ US	■45,000/month	Premium ML-focused

Capacity: A single L4, A10G, or RTX 4090 running Llama 3.1 8B via vLLM batching comfortably serves **30–50 simultaneous requests**. That's enough for one school of this size, with room for several more schools sharing the same server.

Per-school cost when multiple schools share one server (E2E L4 reserved at ■30,000/month):

Schools sharing one server	Cost per school per month
1	■30,000
3	■10,000
5	■6,000

Pros:

- Predictable flat cost regardless of usage spikes
- Full data sovereignty — student PII never leaves our infrastructure
- No per-token billing pressure as usage grows
- Cost per school drops sharply with shared infrastructure
- Independence from third-party AI vendors

Cons:

- Quality gap: open-source models score ~60–70% of Claude Sonnet on complex tasks (essay grading, multi-document summarization)
- High operational burden: model updates, GPU driver maintenance, monitoring, on-call duty
- Fixed cost even when no users are active
- Capacity planning required when usage grows
- Single-server design has a single point of failure (mitigation: keep a managed API as fallback)

Approach 3 — Self-hosted on Bare Metal (own the hardware)

How it works in plain language:

Instead of renting a virtual machine in the cloud, we **buy** the actual physical server with a GPU inside it (a one-time purchase). Then we rent rack space at a colocation data center in India to host the hardware. The data center provides power, cooling, internet bandwidth, and physical security — but the machine is ours.

What we manage: Everything Approach 2 covers, plus the physical hardware (procurement, warranty, replacement of failed parts, lifecycle management). The data center handles power/cooling/connectivity.

Pricing:

Cost type	Amount	Notes
One-time: GPU server (RTX 4090 + workstation)	■2,50,000–4,00,000	Depreciated over 4 years = ~■6,000/month equivalent
Monthly: Colocation rack space (1U with power)	■8,000–15,000/month	At Indian DCs like CtrlS, Yotta, Sify
Monthly: Bandwidth (~100 Mbps committed)	Often included in colo	
Monthly: Effective cost	~■14,000–21,000/month	After hardware amortization

Pros:

- Lowest long-term cost — hardware paid once, only colocation rent thereafter
- Maximum control over hardware specs (can pick exact GPU, RAM, storage)
- Best for very stable, predictable usage
- Strong data residency story (you literally own the hardware, in India)
- Hardware can be re-used for AI training or other workloads off-hours

Cons:

- Significant capital expense upfront (~■3 lakh per server)
- Hardware failures are your problem — when a GPU dies, you ship a new one, ~2–3 day downtime

- Less flexible — can't burst to extra capacity quickly
- Requires somebody on the team who knows hardware
- Lifetime maintenance: warranty management, driver updates, replacement planning
- Not suitable for pilots — you commit to the hardware before knowing demand

Verdict: Strong long-term option if your demand becomes stable and large. **Not recommended for the first 1–2 years** while demand and usage patterns are still evolving.

Approach 4 — Serverless GPU Inference (pay-per-use AI hosting)

How it works in plain language:

You don't rent a GPU and you don't make raw AI API calls — instead, you tell a platform "here's the open-source model I want to run" and they auto-scale GPUs behind the scenes. You're billed per second of GPU time actually used. When nobody's asking questions, you pay nothing.

Providers: **Replicate, Modal, Banana.dev, Together AI, Anyscale.**

Pricing examples (May 2026):

Provider	GPU type	Per-hour cost when active	Effective cost at 30% utilization
Replicate (Llama 3.1 8B serverless)	A10G	\$1.40/hour active	~\$25,000/month at moderate use
Modal (custom deployment)	A10G	\$1.10/hour active	~\$20,000/month
Together AI (managed Llama)	Per token: \$0.20 in / \$0.20 out per M	n/a	~\$6,000/month (similar to OpenRouter)

Pros:

- No fixed cost — pay only when used
- Excellent for spiky, irregular traffic
- No capacity planning needed
- Can scale to thousands of concurrent users automatically

Cons:

- Cold starts: when nobody has used the AI for a few minutes, the next request takes 20–60 seconds because the model has to load. Bad for user experience.
- Effective cost can exceed Approach 2 once usage is consistent
- Less control over the inference environment
- Some providers cache model weights aggressively but most don't, so cold starts are real

Verdict: Useful for development and for products with very irregular traffic (mostly idle, occasional bursts). For a school product that has consistent daily usage during school hours, **not the best fit** — you'd pay continuously while suffering occasional cold-start delays.

Approach 5 — Hybrid (Recommended)

How it works in plain language:

Combine Approach 2 (self-hosted GPU running an open-source model) with Approach 1 (managed AI API kept on standby). A small router inside the School LLM app decides where each question goes:

- **Easy questions** (factual Q&A, simple summaries) → go to our self-hosted Llama 3.1 8B (cheap)
- **Hard questions** (essay grading, multi-page summaries) → go to Claude Haiku API (expensive but accurate)
- **If the self-hosted server is down or overloaded** → automatically fall back to the managed API

Expected split: about **85% of queries handled by self-hosted, 15% by the managed API**.

Per-school monthly cost (one school on its own GPU + 15% Claude usage):

Component	Cost
Hetzner GEX44 GPU server (one server, serves 1–3 schools)	■17,000
Claude Haiku for the 15% "hard" queries (~15,000 calls × 3,500 tokens avg)	~■10,000
Total per school per month	■27,000

Per-school monthly cost (three schools sharing one GPU server):

Component	Cost
Hetzner GEX44 shared across 3 schools	■5,600 per school
Claude Haiku for the 15% "hard" queries	~■10,000
Total per school per month	■15,600

Pros:

- Quality match for the queries that matter (top 15% routed to Claude Haiku)
- Cost roughly 50% below a pure Claude deployment
- Most student data stays on self-hosted infrastructure
- Graceful degradation: if managed API is down, everything falls back to self-hosted (and vice versa)
- Predictable base cost with an elastic margin

Cons:

- Two systems to operate instead of one (modest extra complexity)
- Routing rules need occasional tuning as model capabilities evolve
- Quality monitoring needs to track both paths

5. Cloud GPU Provider Choice — Which to Pick

Within Approach 2 (and within the GPU half of Approach 5), the next decision is **which GPU cloud provider to use**. Here's the comparison with a clear recommendation.

Indian providers (best for our market)

E2E Networks is an Indian-owned cloud with data centers in India. Pros: low latency for Indian users, INR billing, GST-compliant invoices, Indian support team, data residency for student records. Pricing is competitive — about half of AWS Mumbai for similar hardware. This is the **recommended provider** for an Indian-customer-focused product.

GPU	Spec	E2E on-demand	E2E reserved (1-year)
L4	24 GB VRAM	■60/hour ≈ ■43,000/month	~■30,000/month
L40S	48 GB VRAM	■130/hour ≈ ■93,000/month	~■65,000/month
A100 40GB	40 GB VRAM	■170/hour ≈ ■1,22,000/month	~■85,000/month

Global hyperscalers (premium reliability)

AWS, Google Cloud, Microsoft Azure all have Indian regions and offer GPU instances. Best when you absolutely need 99.99% uptime, advanced compliance certifications, or enterprise support contracts. **Cost is 50–80% higher than E2E for the same hardware.**

Provider	Instance	India region	Monthly cost (1-yr reserved)
AWS	g5.xlarge (A10G)	Mumbai	■39,000
AWS	g6.xlarge (L4)	Mumbai	■35,000 (estimated)
Google Cloud	g2-standard-4 (L4)	Mumbai	~■38,000
Microsoft Azure	NC4as_T4_v3	Pune	~■28,000

Cheap European providers (lowest cost)

Hetzner is the cheapest reliable GPU host worldwide. Trade-off: data center is in Germany/Finland, ~150 ms higher latency from India. For non-time-critical AI workloads (responses already take 2–5 seconds), the latency is acceptable, but it's a downside for an Indian-market product.

Provider	Server	Monthly cost	India region
Hetzner	GEX44 (RTX 4000 Ada 20 GB)	■17,000	■
Hetzner	GEX130 (RTX A6000 48 GB)	~■45,000	■

GPU-specialist clouds (cheap but variable reliability)

RunPod, Vast.ai, Lambda Labs built their business around GPU rentals for ML training. Production-grade hosting is a secondary use case.

Provider	GPU	Hourly	Monthly (24/7)	Notes
RunPod (Secure Cloud)	RTX 4090	\$0.34/hr	■20,000	Decent for production
Vast.ai (marketplace)	RTX 4090	\$0.27/hr	■16,000	Cheapest, but instances can disappear
Lambda Labs	A10G	\$0.75/hr	■45,000	Premium ML focus

Provider recommendation

Stage	Recommended provider	Why
Pilot (no GPU needed)	None — use Claude API directly	No GPU investment until you have demand
Growing (3–10 schools or 10K individual users)	E2E Networks L4 reserved (■30K/month)	Indian DC, INR billing, good price, decent reliability
Mature (15+ schools or 50K+ users)	E2E Networks (multiple servers) OR migrate to AWS Mumbai g5.xlarge	When uptime SLAs become customer-facing requirements
Enterprise / mission-critical	AWS Mumbai with multi-AZ redundancy + AWS support contract	When schools sign SLA contracts

Avoid: Vast.ai for production (instances can be reclaimed by their owners). Lambda Labs unless you specifically need their ML platform features. Bare-metal purchases in year 1 (premature commitment).

6. Side-by-side Cost Comparison

Per-school cost at different scales (INR per month)

Schools served	Approach 1: Claude only	Approach 2: Self-hosted on E2E L4	Approach 5: Hybrid (Hetzner GEX44 + Claude Haiku)
1 school	■60,000	■30,000 (one dedicated server)	■27,000
3 schools	■1,80,000 (■60K × 3)	■30,000 (one server shared)	■47,000 (one server shared + Claude)
5 schools	■3,00,000	■30,000 (one server, near capacity)	■67,000
10 schools	■6,00,000	■60,000 (two servers)	■1,14,000
25 schools	■15,00,000	■1,50,000 (5 servers)	■2,67,000

Quality, latency, and operational dimensions

Dimension	Claude API	Self-hosted	Hybrid
Response quality (subjective)	10/10	6.5/10	9/10
Median response latency	1.5 s	2.5 s	2.0 s
Data residency	Vendor cloud	Self-controlled	Mostly self-controlled
Time to onboard a new school	Minutes	Hours	Hours
Operational complexity	Low	High	Medium
Cost predictability	Variable (usage-driven)	Fixed	Mostly fixed
Vendor concentration risk	High	None	Low

7. Recommendation — Phased Approach

The right strategy is not to pick one deployment approach and stick with it forever. The right strategy is to **start simple, then evolve as scale justifies more sophisticated infrastructure**.

Phase 1 — Pilot (months 1–2)

- **Use Approach 1: Managed API (Claude) directly.**
- Onboard the first 1–2 schools.
- One Claude API key per school for billing and quota visibility.
- **Why:** Zero infrastructure investment, fastest time to market, highest quality so the product impresses early customers. The per-school cost (~\$60,000/month) is acceptable for 1–2 schools while you validate product-market fit.
- **Monthly cost:** \$60,000 to \$1,20,000.

Phase 2 — Hybrid rollout (months 3–6)

- **Transition to Approach 5: Hybrid.**
- Provision one **E2E Networks L4 GPU server in Mumbai** (reserved, \$30,000/month) OR a **Hetzner GEX44** (\$17,000/month) if cost is the priority.
- Deploy **vLLM serving Llama 3.1 8B** on it.
- Implement the router: simple queries → self-hosted Llama, complex queries → Claude Haiku API.
- Migrate existing pilot schools to the hybrid stack and onboard 3–5 more.
- **Why:** Cost per school drops dramatically (from \$60K to ~\$15-27K), quality is preserved on the queries that matter, and you build the operational muscle to run your own GPU infrastructure.
- **Monthly cost:** \$50,000 to \$1,00,000 (3–5 schools sharing one server).

Phase 3 — Scale (months 7–12)

- Add a second GPU server when sustained concurrent usage crosses ~70% of the first one.
- Consider upgrading from Hetzner to AWS Mumbai g5.xlarge or E2E L4 for in-region latency now that the customer base justifies the spend.
- Re-evaluate the router rules monthly using quality metrics from real student usage.
- **Monthly cost:** ■2,00,000 to ■3,00,000 for 10 schools (vs ~■6,00,000 on Claude-only).

Phase 4 — Sovereign (year 2+, optional)

- If 20+ schools are active and data residency becomes a contractual requirement, evaluate replacing the Claude Haiku 15% with a larger self-hosted model (Llama 3.3 70B on a higher-tier GPU).
 - This removes the last external API dependency at the cost of one more GPU server.
 - Also evaluate **Approach 3 (bare metal in Indian colocation)** at this stage — the demand is now stable enough to justify the capex.
-

8. Risks and Mitigations

■ Phase 2 (3-10 schools):	Hybrid on E2E or Hetzner	■
■ Phase 3 (10+ schools):	Hybrid with 2+ GPU servers	■
■ Phase 4 (year 2+, optional):	Sovereign self-hosted only	■

Appendix A — Pricing Sources (verified May 2026)

- **Anthropic Claude API:** Haiku 4.5 at \$1/\$5 per million input/output tokens; Sonnet 4.6 at \$3/\$15; prompt caching at 1.25× write / 0.10× read of base input rate.
- **Google Gemini 2.5 Flash:** \$0.30 input / \$2.50 output per million tokens.
- **OpenRouter Llama 3.3 70B:** \$0.10 input / \$0.32 output per million tokens. Free tier available but rate-limited.
- **E2E Networks GPU cloud (India):** L4 at ■50–70/hour, A100 40GB at ■170/hour, L40S at ■120–150/hour.
- **AWS EC2 g5.xlarge:** \$1.006/hour on-demand; \$0.634/hour 1-year reserved; \$0.435/hour 3-year reserved.
- **AWS EC2 g6.xlarge:** \$0.8048/hour on-demand (NVIDIA L4 GPU).
- **Hetzner GEX44:** €184/month (NVIDIA RTX 4000 SFF Ada, 20 GB VRAM, i5-13500, 64 GB RAM, 2×1.92 TB NVMe).
- **RunPod RTX 4090:** \$0.34–0.69/hour Secure Cloud.
- **Vast.ai RTX 4090:** \$0.27/hour marketplace.
- **Lambda Labs A10G:** \$0.75/hour.

Appendix B — Workload Model Assumptions

- 100,000 LLM calls per school per month
- 2,500 input + 600 output tokens per call average
- 70% lightweight (Q&A, evaluation) / 30% complex (summaries, grading) model split
- 60% of input volume benefits from prompt caching
- 60% teacher daily-active rate, 50% student daily-active rate
- 24 active days per month (weekdays + light weekend use)
- A single L4 or A10G or RTX 4090 serves ~30–50 concurrent requests under vLLM batching

Appendix C — Glossary

- **Token:** The unit of text the AI bills on. Roughly 0.75 words = 1 token.
- **Prompt caching:** Re-using a long system prompt or PDF context across multiple queries at 10% of normal input cost.
- **vLLM:** A high-throughput inference server that batches concurrent requests; 3–10× more efficient than naive single-request serving.

- **Ollama:** A simpler model-serving runtime, good for development and small deployments.
- **Quantization (Q4_K_M):** A compression technique that lets large models run on smaller GPUs with minor quality loss.
- **Colocation:** Renting space in a data center to house your own hardware.
- **Serverless GPU:** A model where you pay per second of GPU time used, not per server rented.
- **Cold start:** The 20–60 second delay when an AI model has to be loaded into GPU memory before answering its first question.
- **Bare metal:** Owned physical servers (not rented virtual machines).
- **Spot instance:** Cheap cloud capacity that can be reclaimed by the provider on short notice; not suitable for always-on services.